



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
BRASÍLIA

TECNÓLOGO SISTEMAS PARA INTERNET

Professor: Gustavo Henrique Dornelas de Deus

Lista de Exercícios do Aluno

Desenvolva os **algoritmos**, dentre os apresentados nessa lista de exercícios, usando pelo menos uma das técnicas: português estruturado ou JAVA.

1. Classe Calculadora

Objetivo: Desenvolver uma classe Calculadora que realize operações matemáticas básicas utilizando três atributos numéricos.

Requisitos:

- **Atributos:**
 - valor1: Número do tipo double.
 - valor2: Número do tipo double.
 - valor3: Número do tipo double.
- **Construtores:**
 - **Construtor Padrão:** Inicializa valor1, valor2 e valor3 com zero.
 - **Construtor com um Parâmetro:** Inicializa valor1 com o valor fornecido; valor2 e valor3 são definidos como zero.
 - **Construtor com Dois Parâmetros:** Inicializa valor1 e valor2 com os valores fornecidos; valor3 é definido como zero.
 - **Construtor com Três Parâmetros:** Inicializa valor1, valor2 e valor3 com os valores fornecidos.
- **Métodos:**
 - **Soma:** Método que retorna a soma de valor1, valor2 e valor3.
 - **Subtração:** Método que retorna a subtração de valor1 menos valor2 e menos valor3.
 - **Multiplicação:** Método que retorna o produto de valor1, valor2 e valor3.
 - **Divisão:** Método que retorna a divisão de valor1 por valor2 e por valor3. Deve verificar se valor2 e valor3 são diferentes de zero antes de realizar a operação, evitando erro de divisão por zero.
- **Método main:**
 - Criar instâncias da classe Calculadora utilizando diferentes construtores.
 - Demonstrar o uso dos métodos de operação, exibindo os resultados no console.

```

package model;

public class Calculadora {
    public double valor1;
    public double valor2;
    public double valor3;

    public Calculadora(double valor1, double valor2, double valor3) {
        this.valor1 = valor1;
        this.valor2 = valor2;
        this.valor3 = valor3;
    }

    public Calculadora() {
        this.valor1 = 0;
        this.valor2 = 0;
        this.valor3 = 0;
    }

    public Calculadora(double valor1) {
        this.valor1 = valor1;
        this.valor2 = 0;
        this.valor3 = 0;
    }

    public Calculadora(double valor1, double valor2) {
        this.valor1 = valor1;
        this.valor2 = valor2;
        this.valor3 = 0;
    }

    public double soma() {
        return this.valor1 + this.valor2 + this.valor3;
    }

    public double subtracao() {
        return this.valor1 - this.valor2 - this.valor3;
    }

    public double multiplicacao() {
        return this.valor1 * this.valor2 * this.valor3;
    }

    public double divisao() {
        if(this.valor2 == 0 || this.valor3 == 0) {
            System.out.println("Erro escolha outro número maior que 0.");
            return Double.NaN;
        }
        return (this.valor1 / this.valor2) / this.valor3;
    }

    @Override
    public String toString() {
        return "Calculadora [valor1=" + valor1 + ", valor2=" + valor2 + ", valor3=" + valor3 + "]" + " | Soma: "
        + soma() + " | Subtração: " + subtracao() + " | Multiplicação: " + multiplicacao() +
        " | Divisão: " + (valor2 == 0 || valor3 == 0 ? "Erro (Divisão por zero)" : divisao());
    }
}

```

```

package main;

import model.Calculadora;

public class Main {

    public static void main(String[] args) {
        System.out.println("Testando os diferentes métodos: ");
        Calculadora cal1 = new Calculadora(25.4,99.0,4.0);
        Calculadora cal2 = new Calculadora(66.5,42.8);
        Calculadora cal3 = new Calculadora(30.0);
        Calculadora cal4 = new Calculadora();

        System.out.println(cal1);
        System.out.println(cal2);
        System.out.println(cal3);
        System.out.println(cal4);
    }

}

```

2. Classes Endereço e Pessoa

Objetivo: Implementar classes que representem um endereço e uma pessoa, estabelecendo uma relação entre elas.

Classe Endereco:

- **Atributos:**
 - logradouro: String que representa o nome da rua ou avenida.
 - numero: Número da residência ou estabelecimento.
 - complemento: Informações adicionais sobre o endereço (opcional).
 - bairro: Bairro onde o endereço está localizado.
 - cidade: Cidade do endereço.
 - cep: Código de Endereçamento Postal.
- **Construtores:**
 - **Construtor Vazio:** Inicializa os atributos com valores padrão (por exemplo, strings vazias ou zero).
 - **Construtor Completo:** Inicializa todos os atributos com os valores fornecidos.
- **Método:**
 - **toString:** Retorna uma representação em String do endereço, apresentando todos os atributos de forma legível.

Classe Pessoa:

- **Atributos:**
 - nome: Nome completo da pessoa.
 - cpf: Cadastro de Pessoa Física.
 - endereco: Objeto da classe Endereco, representando o endereço da pessoa.
- **Construtores:**
 - **Construtor Vazio:** Inicializa os atributos com valores padrão.
 - **Construtor Completo:** Inicializa nome, cpf e endereco com os valores fornecidos.
- **Método:**
 - **toString:** Retorna uma representação em String da pessoa, incluindo nome, CPF e endereço.
- **Método main:**
 - Criar instâncias das classes Endereco e Pessoa.
 - Demonstrar a criação de objetos e a utilização do método toString para exibir as informações no console.

```

package model;

public class Endereco {
    public String logradouro;
    public int numero;
    public String complemento;
    public String bairro;
    public String cidade;
    public String CEP;

    public Endereco() {
    }

    public Endereco(String logradouro, int numero, String complemento, String bairro, String cidade,
        String CEP) {
        this.logradouro = logradouro;
        this.numero = numero;
        this.complemento = complemento;
        this.bairro = bairro;
        this.cidade = cidade;
        this.CEP = CEP;
    }

    @Override
    public String toString() {
        return "Endereco [logradouro=" + logradouro + ", numero=" + numero + ", complemento=" + complemento
            + ", bairro=" + bairro + ", cidade=" + cidade + ", CEP=" + CEP + "];"
    }
}

```

```

package model;

public class Pessoa {
    public String nome;
    public String CPF;
    public Endereco endereco;

    public Pessoa() {
    }

    public Pessoa(String nome, String CPF, Endereco endereco) {
        this.CPF = CPF;
        this.nome = nome;
        this.endereco = endereco;
    }

    @Override
    public String toString() {
        return "Pessoa [nome=" + nome + ", CPF=" + CPF + ", " + endereco + "];"
    }
}

```

```

package main;

import model.Endereco;
import model.Pessoa;

public class Main {

    public static void main(String[] args) {
        Endereco endereco = new Endereco("SQS", 308, "casa", "Asa Sul", "Brasília", "4582258");

        Pessoa pessoa = new Pessoa("Alessandra Rodrigues", "466666464800", endereco);

        System.out.println(endereco);
        System.out.println(pessoa);
    }
}

```

3. Classe Conta

Objetivo: Desenvolver uma classe Conta que simule operações bancárias básicas.

Atributos:

- numero: Número da conta bancária.
- saldo: Saldo atual da conta.
- limite: Limite de crédito disponível para saque.
- nome: Nome do titular da conta.

Construtores:

1. **Construtor Padrão:** Inicializa os atributos com valores padrão.
2. **Construtor com Parâmetros:** Permite a inicialização dos atributos com valores específicos fornecidos.

Métodos:

1. **Sacar:** Subtrai um valor do saldo, desde que o valor não ultrapasse o limite de saque diário.
2. **Depositar:** Adiciona um valor ao saldo.
3. **Transferir:** Transfere um valor para outra conta, realizando a subtração do saldo da conta atual e a adição na conta destino, desde que o valor não ultrapasse o limite de saque diário.

Método main:

- Criar instâncias da classe Conta.
- Demonstrar a criação de contas e a execução das operações de saque, depósito e transferência, exibindo os resultados no console.

Conta
+numero: int +saldo: double +limite: double +nome: String
+saca(valor: double): boolean +deposita(valor: double) +transfere(destino: Conta, valor: double)

```

package model;

public class Conta {
    public int numero;
    public double saldo;
    public double limite;
    public String nome;

    public Conta() {
        this.numero = 0;
        this.saldo = 0;
        this.limite = 0;
        this.nome = "sem nome";
    }

    public Conta(int numero, double saldo, double limite, String nome) {
        this.numero = numero;
        this.saldo = saldo;
        this.limite = limite;
        this.nome = nome;
    }

    public boolean sacar(double valor) {
        if( valor > this.limite) {
            System.out.println("Erro: O valor que quer sacar ultrapassa o limite de crédito disponível para saque diário.");
            return false;
        }
        if( valor > this.saldo) {
            System.out.println("Erro: saldo insuficiente para sacar o valor digitado");
            return false;
        }
        this.saldo -= valor;
        System.out.println("Saque de R$: " + valor+ " realizado com sucesso.");
        return true;
    }

    public void depositar(double valor) {
        if (valor > 0) {
            this.saldo += valor;
            System.out.println("Depósito de R$ " + valor + " realizado com sucesso na conta de " + this.nome);
        } else {
            System.out.println("Erro: O valor do depósito deve ser maior que zero.");
        }
    }

    public void transferir(double valor, Conta contaDestino) {
        System.out.println("Iniciando transferência de " + this.nome+ " para " + contaDestino.nome+ "...");

        if(this.sacar(valor)) {
            contaDestino.depositar(valor);
            System.out.println("Transferência de R$ " + valor+ " concluída com sucesso.");
        }
        else {
            System.out.println("Erro: A transferência não pôde ser realizada.");
        }
    }

    @Override
    public String toString() {
        return "Conta [numero=" + numero + ", saldo=" + saldo + ", limite=" + limite + ", nome=" + nome + "];"
    }
}

```

```
package main;

import model.Conta;

public class Main {
    public static void main(String[] args) {
        System.out.println("Criação de contas:");
        Conta conta1 = new Conta(658744, 365.8, 5000.0, "Alessandra R");
        Conta conta2 = new Conta(445888, 745.5, 400, "Bruno B");
        Conta conta3 = new Conta(422631, 85.9, 800.5, "Cora C");
        System.out.println(conta1);
        System.out.println(conta2);
        System.out.println(conta3);

        System.out.println("Operações: ");
        conta2.depositar(75);
        System.out.println(conta2);

        conta1.sacar(400);
        System.out.println(conta1);

        conta3.transferir(80, conta1);
        System.out.println(conta3);
        System.out.println(conta1);
    }
}
```